Medical Scribe System — KT Runbook (EC2 GPU + S3)

Purpose: This runbook documents the end-to-end architecture, components, data flow, deployment, and testing for the Scribe system that turns doctor–patient audio into Transcript, SOAP note, Summary, and Structured Note, and stores artifacts in Amazon S3.

1) What this system does:

This system converts a **doctor–patient audio recording** into a set of clinical documentation artifacts and stores everything in **Amazon S3** in a predictable folder structure.

Inputs:

- An audio file (mp3/m4a/webm/wav, etc.)
- Encounter identifiers and context:
  - organisation_id, clinic_id, doctor_id, patient_id
  - provider (source channel, app name, device type, etc.)
  - Optional display labels like organisation_name, clinic_name
  - Optional output preference like format_type=soap

Processing steps (what actually happens):

1. **Audio ingestion**
   - Backend receives audio via POST /process-audio (multipart form).

2. **Raw audio persistence**
   - The raw file is uploaded to S3 as audio.mp3 (or original extension), with metadata attached.

3. **Audio decoding / normalization**
   - ffmpeg converts arbitrary formats into a consistent format (PCM WAV, correct sample rate/channels) that ASR models like Whisper handle reliably.

4. **Speech-to-text (ASR)**
   - A **GPU-backed Whisper pipeline** transcribes the decoded audio into a text transcript.

5. **Clinical formatting**
   - The transcript is sent to **OpenAI** to produce:
     - **SOAP note** (soap.md)
     - **Summary** (summary.txt)
     - **Structured note** (note.txt) (often closer to “final clinical note” format)

6. **Artifact persistence**
   - All outputs are written back to the **same S3 encounter folder** with the **same metadata**.

7. **API response**
    - API returns:
        - Inline text (transcript + SOAP + summary + note)
        - S3 keys/paths for each artifact (so frontend can fetch later)

What you end up with

- A single encounter folder in S3 that contains the complete set:
    - raw audio
    - transcript
    - clinician-ready documentation outputs
- Consistent metadata across objects for traceability and searching.

---

2) Why we built it:

A) Saves doctor time on documentation:

- Clinical documentation (SOAP + structured note) is a major time drain.
- The system automates:
    - transcription
    - formatting into clinical structures
    - producing a readable "ready-to-review" note

B) Operational simplicity (S3 is the source of truth):

Instead of building a complex DB-driven pipeline first, we use S3 as the "truth layer":

- **Every encounter becomes a folder**
- **Every artifact is a file**
- **Every file carries consistent metadata**
  This makes the system easy to:
- audit
- replay
- debug
- extend later with a DB if needed

C) Speed + cost control:

- **Whisper on GPU (EC2 g4dn.xlarge / Tesla T4)** is fast for transcription and keeps ASR workload inside our infra.
- OpenAI is used only for **text transformation** (clinical formatting), which is typically lighter than full audio processing.

D) Easy integration pattern

The integration contract is simple:

- Frontend uploads audio → backend returns:
    - generated text
    - S3 keys for storage
      So the UI can:
- show results instantly
- store/retrieve later using keys

---

3) High-level architecture:

Frontend → FastAPI (EC2 GPU) → ffmpeg → Whisper GPU → OpenAI → S3

```
┌──────────────────────────────────────────────────────────────────────┐
│                        Client Layer (Web / Mobile)                   │
│  • Records or selects an audio file                                  │
│  • Collects encounter context (organisation_id, clinic_id, doctor_id, etc.)  │
│  • Sends request to backend                                          │
└──────────────────────────────────────────────────────────────────────┘
                              │
                              │  HTTP (multipart/form-data)
                              │  POST /process-audio
                              │  Body: file + IDs + provider + format_type
                              v
┌──────────────────────────────────────────────────────────────────────┐
│              Compute Layer: EC2 GPU (Ubuntu 22.04, g4dn.xlarge, Tesla T4)  │
│                 FastAPI + Uvicorn (Python venv, systemd managed)     │
│                                                                      │
│  Step 1 — Persist Raw Input                                          │
│    • Upload original audio to S3 as: audio.mp3                       │
│    • Attach metadata headers (org/clinic/doctor/patient/provider/date/ts)  │
│                                                                      │
│  Step 2 — Decode / Normalize Audio (ffmpeg)                          │
│    • Convert mp3/m4a/webm -> PCM WAV (stable format for ASR)         │
│                                                                      │
│  Step 3 — ASR (Speech-to-Text)                                       │
│    • Whisper on GPU (fp16 inference)                                 │
│    • Output: transcript.txt                                          │
│                                                                      │
│  Step 4 — Clinical Formatting (LLM)                                  │
│    • Send transcript to OpenAI                                       │
│    • Generate: soap.md, summary.txt, note.txt                        │
│                                                                      │
│  Step 5 — Persist Outputs                                            │
│    • Upload all artifacts to the same S3 encounter folder            │
│    • Keep the same metadata on each object                           │
└──────────────────────────────────────────────────────────────────────┘
                              │
                              │  boto3 (S3 put_object / get_object)
                              v
┌──────────────────────────────────────────────────────────────────────┐
│                    Storage Layer: Amazon S3 (Source of Truth)        │
│  Encounter folder contains:                                          │
│    • audio.mp3                                                       │
│    • transcript.txt                                                  │
│    • soap.md                                                         │
│    • summary.txt                                                     │
│    • note.txt                                                        │
│  Each object stores metadata headers:                                │
│    organisation-id, clinic-id, doctor-id, patient-id, year, month, date,  │
│    provider, ts                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

S3 folder strategy (why it matters):
We store each encounter as a deterministic folder path so you can locate artifacts without a DB.

Example structure:

```
s3://<bucket>/
  <organisation_id>/<clinic_id>/<doctor_id>/
    YYYY/MM/DD/<patient_id>/enc_YYYYMMDD_HHMMSS/
      audio.mp3
      transcript.txt
      soap.md
      summary.txt
      note.txt
```

Objects stored (exact)

- audio.mp3 (raw input)
- transcript.txt (ASR output)
- soap.md (SOAP note)
- summary.txt (short summary)
- note.txt (structured note / final note)

Metadata on each object (why + how) -
Each S3 object includes metadata (headers) like:

- organisation-id
- clinic-id
- doctor-id
- patient-id
- year, month, date
- provider
- ts (timestamp)

Why metadata is important

- Traceability: every artifact is tied to a specific encounter context.
- Debugging: you can track mismatches and pipeline issues quickly.
- Future: easy indexing later (e.g., S3 Inventory → Athena → search).

---

4) Services & components (hardcore: what + where + why):

EC2 (Ubuntu 22.04) on g4dn.xlarge (Tesla T4)

What it is
- A GPU-enabled virtual machine.

Where used
- Runs the backend API and Whisper inference.

Why
- Whisper transcription is much faster on GPU.
- Predictable performance and single-box simplicity for an MVP.

FastAPI (Uvicorn) + Python venv

What
- FastAPI: web framework (API endpoints).
- Uvicorn: ASGI server to run FastAPI.
- venv: isolates Python dependencies from OS packages.

Where
- Inside EC2 instance.

Why
- Simple, modern API stack.
- Clean dependency control with venv (repeatable deployments).

CUDA / NVIDIA drivers

What
- GPU runtime required for PyTorch to use Tesla T4.

Where
- Installed at OS level on EC2.

Why
- Without drivers/CUDA, Whisper runs on CPU and becomes slow.

Whisper HF pipeline (fp16)

What
- Whisper model served via Hugging Face / PyTorch pipeline.
- fp16 means half-precision inference (faster, lower VRAM usage).

Where
- EC2 GPU instance.

Why
- Fast transcription at lower operational cost than managed speech services (depending on workload).

---

ffmpeg (system package)
What
- Audio decoding + conversion tool.

Where
- OS package installed via apt.

Why
- Users upload audio in many formats.
- Whisper and audio libraries behave best when input is normalized:
    - correct codec
    - consistent sample rate
    - mono vs stereo handled
    - avoids “model errors” caused by weird encodings

OpenAI (gpt-4o-mini) for SOAP/Summary/Note

What
- Text generation/formatting model.

Where
- Called from backend over HTTPS API.

Why
- Clinical formatting is not just summarization—needs structure:
    - SOAP headings
    - concise medical phrasing
    - consistent templates
      This is a strong use case for an LLM after transcript is produced.

Amazon S3 with metadata

What
- Object storage bucket.

Where
- AWS.

Why
- Durable storage for:
    - raw audio
    - transcript
    - clinical artifacts
- Becomes the “truth layer” without adding DB complexity at first.

---

systemd service (scribe-api.service)

What
- Linux service manager.

Where
- EC2 OS.

Why
- Keeps the API running across:
  - reboots
  - crashes
  - deployments
- Gives standard operational controls:
  - start/stop/restart
  - logs
  - auto-start

Optional ngrok (HTTPS public URL)

What
- Tunnels local/EC2 port to a public HTTPS endpoint.

Where
- Runs on EC2 if enabled.

Why
- Useful when frontend needs HTTPS callback to EC2 without DNS/ALB setup.
- Great for demos and quick integrations.

Logs via journalctl

What
- systemd log viewer.

Where
- On EC2.

Why
- Single command to see live backend logs:
  - request failures
  - ffmpeg errors
  - model loading issues
  - OpenAI/S3 exceptions

---

5) Data flow (detailed + diagram):

End-to-end flow:

1. Frontend sends:
    - audio file
    - IDs (organisation_id, clinic_id, etc.)
    - optional format_type
2. API writes raw audio.mp3 to S3 with metadata
3. API calls ffmpeg to decode → PCM/WAV
4. Whisper GPU generates transcript.txt
5. Transcript is passed to OpenAI → generates:
    - soap.md
    - summary.txt
    - note.txt
6. API stores each artifact in the same S3 encounter folder with same metadata
7. API returns:
    - inline text results
    - S3 object keys

Data flow diagram:

```
[Frontend]
   |
   | POST /process-audio (audio + IDs)
   v
[FastAPI on EC2 GPU]
   |
   | (A) S3 put_object: audio.mp3 + metadata
   v
[ S3 /encounter-folder/audio.mp3 ]
   ^
   |
   | (B) ffmpeg decode (local)
   v
[PCM/WAV temp file]
   |
   | (C) Whisper GPU inference
   v
[transcript text]
   |
   | (D) OpenAI formatting
   v
[soap / summary / note text]
   |
   | (E) S3 put_object: transcript.txt, soap.md, summary.txt, note.txt + metadata
   v
[ S3 encounter folder complete ]
```

---

6) Deployment & operations (why each install exists):

Install ffmpeg

```
sudo apt-get update -y && sudo apt-get install -y ffmpeg
```

Why

- Required for decoding real-world audio inputs reliably.
- Prevents failures on formats like m4a/webm or odd encodings.

Python dependencies (what each one does):

- **torch, torchaudio**
  - GPU compute + audio helpers; foundation for Whisper inference

- **transformers, accelerate**
  - Model loading/inference pipeline + performance helpers (device placement, fp16 usage)

- **ffmpeg-python**
  - Programmatic control of ffmpeg from Python (or wrapper integration)

- **fastapi**
  - API framework (routing, request parsing, validation)

- **uvicorn**
  - Runs the FastAPI service in production mode

- **python-multipart**
  - Required for multipart/form-data file uploads

- **pydantic**
  - Strong request/response schemas and validation

- **boto3**
  - S3 read/write, metadata headers, listing, etc.

- **numpy**
  - Audio arrays and numeric utilities often needed around preprocessing

- **python-dotenv**
  - Loads environment variables from .env during dev

- **openai**
  - Calls OpenAI to generate SOAP/summary/note

Start/restart & logs (what each command accomplishes)

```
sudo systemctl daemon-reexec && sudo systemctl daemon-reload
```

Reloads systemd's understanding of unit files after edits.

```
sudo systemctl enable --now scribe-api
```

Enables auto-start at boot and starts immediately.

```
sudo systemctl restart scribe-api
```

Restarts the API to apply code/config changes.

```
sudo systemctl status scribe-api
```

Shows service state, recent failures, exit codes.

```
sudo journalctl -u scribe-api -f
```

Local / manual testing:

```
curl -X POST "http://44.246.211.74:8000/process-audio" \
  -F "file=@\"C:/Users/Shivang Sharma/Desktop/okay.mp3\"" \
  -F "organisation_id=UGZP001" \
  -F "clinic_id=AFC001" \
  -F "doctor_id=DOC567" \
  -F "patient_id=HIOKHM" \
  -F "provider=king" \
  -F "organisation_name=Primary Care Associates" \
  -F "clinic_name=Sunrise Clinic" \
  -F "format_type=soap" \
  -o response.json \
&& cat response.json
```

- -X POST → calls the API endpoint.
- -F → sends multipart form data:
  - file=@... attaches the audio file.
  - other -F fields are plain text form fields.
- -o response.json saves the HTTP response body into response.json.
- && cat response.json prints it.

Why this is best

- Avoids security group/public IP networking issues.
- Confirms API+ffmpeg+GPU+S3 work internally.

B) Health check first (always)

curl http://44.246.211.74:8000/healthz

You want:

- ok: true
- bucket, region
- GPU/CUDA shown (if your healthz prints it)

C) Debug request failures

If it fails, immediately tail logs:

sudo journalctl -u scribe-api -f

This usually tells you exactly:

- ffmpeg path issue
- model load error
- CUDA not available
- OpenAI key missing
- S3 permission denied